21.2 扩展 API 表面

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~780k tokens，输出 ~70k tokens（本章合计）

上一节我们了解了插件从发现到加载的完整流程。插件一旦被加载，就通过 `OpenClawPluginApi` 这个"窗口"与 OpenClaw 主程序交互。本节将深入分析这个 API 表面的三个层次：内部扩展 API（`extensionAPI.ts`）、面向外部的插件 SDK（`plugin-sdk/index.ts`）、以及插件钩子接口。

21.2.1 `src/extensionAPI.ts` — 扩展可访问的 API 集

内部扩展 API 的定位

`extensionAPI.ts` 是一个**内部再导出模块**（re-export module），它从 OpenClaw 核心模块中精选出扩展可能需要的函数和类型，统一暴露给内部扩展使用：

```
// src/extensionAPI.ts（完整）
export { resolveAgentDir, resolveAgentWorkspaceDir } from "./agents/agent-scope.ts";
export { DEFAULT_MODEL, DEFAULT_PROVIDER } from "./agents/defaults.ts";
export { resolveAgentIdentity } from "./agents/identity.ts";
export { resolveThinkingDefault } from "./agents/model-selection.ts";
export { runEmbeddedPiAgent } from "./agents/pi-embedded.ts";
export { resolveAgentTimeoutMs } from "./agents/timeout.ts";
export { ensureAgentWorkspace } from "./agents/workspace.ts";
export {
  resolveStorePath, loadSessionStore,
  saveSessionStore, resolveSessionFilePath,
} from "./config/sessions.ts";
```

这个文件目前只导出了 **Agent 运行时**和**会话存储**相关的少量接口。它的职责是为**内置扩展**（`extensions/` 目录下与主包一起发布的模块）提供直接访问内部模块的能力。

> **衍生解释**：**Re-export 模式**是 TypeScript/JavaScript 中常见的模块组织模式。一个模块不包含任何逻辑，只通过 `export { X } from "Y"` 语句从其他模块中"挑选"符号重新导出。这样做的好处是：(1) 调用方只需导入一个模块而非散落在多处的实现；(2) 模块作者可以精确控制哪些 API 对外可见，形成稳定的公共接口边界。

与 Plugin SDK 的关系

`extensionAPI.ts` 和 `plugin-sdk/index.ts` 看似都是"导出 API"，但定位不同：

维度	`extensionAPI.ts`	`plugin-sdk/index.ts`
受众	内置扩展（同仓库）	外部第三方插件
导入方式	`import { ... } from "../extensionAPI.js"`	`import { ... } from "openclaw/plugin-sdk"`
稳定性	可随主包版本变化	对外承诺的稳定 API
范围	Agent 运行时、会话存储	通道类型、配置 Schema、工具辅助函数等

内置扩展（如 `extensions/discord/` 、 `extensions/slack/` ）因为和主包在同一个仓库中，可以直接引用内部模块路径；而第三方插件只能通过 `openclaw/plugin-sdk` 这个包名导入，由 jiti 的 alias 机制映射到实际文件。

21.2.2 插件 SDK（ `src/plugin-sdk/` ）

SDK 的设计哲学

`plugin-sdk/index.ts` 是一个 **375 行的巨型再导出模块**，它是外部插件开发者与 OpenClaw 交互的唯一公共 API。这个设计遵循了"窄接口、宽实现"的原则——内部实现可以自由重构，只要 SDK 表面保持稳定，第三方插件就不会因内部变更而失效。

SDK 导出分类

SDK 导出的内容可以分为以下几类：

（1）通道适配器类型

这是 SDK 中最大的一块，导出了通道插件开发所需的全部类型定义：

```
// src/plugin-sdk/index.ts
export type {
  ChannelPlugin,                // 通道插件主类型
  ChannelConfigSchema,          // 配置 Schema 类型
  ChannelConfigAdapter,         // 配置适配器
  ChannelOutboundAdapter,       // 出站消息适配器
  ChannelSecurityAdapter,       // 安全策略适配器
  ChannelSetupAdapter,          // 初始化适配器
  ChannelPairingAdapter,        // 配对适配器
  ChannelGroupAdapter,          // 群组适配器
  ChannelMentionAdapter,        // @提及适配器
  ChannelStatusAdapter,         // 状态监控适配器
  ChannelGatewayAdapter,        // Gateway 集成适配器
  ChannelAuthAdapter,           // 认证适配器
  ChannelElevatedAdapter,       // 权限提升适配器
  ChannelCommandAdapter,        // 命令适配器
  ChannelStreamingAdapter,      // 流式传输适配器
  ChannelThreadingAdapter,      // 线程/话题适配器
  ChannelMessagingAdapter,      // 消息传递适配器
  ChannelDirectoryAdapter,      // 通讯录适配器
  ChannelResolverAdapter,       // 目标解析适配器
  ChannelHeartbeatAdapter,      // 心跳检测适配器
  ChannelMessageActionAdapter,  // 消息操作适配器
  // ... 以及各适配器所需的上下文类型
} from "../channels/plugins/types.js";
```

这些类型组成了第 11 章介绍的通道适配器抽象层的完整接口。

（2）插件核心类型

插件注册和服务管理相关的核心类型：

```
export type {
  OpenClawPluginApi,            // 插件注册 API（最核心的接口）
  OpenClawPluginService,        // 后台服务定义
  OpenClawPluginServiceContext,  // 服务运行上下文
} from "../plugins/types.js";

export type { PluginRuntime } from "../plugins/runtime/types.js";
export type { OpenClawConfig } from "../config/config.js";
export type { ChannelDock } from "../channels/dock.js";
```

（3）配置 Schema 构建器

为各通道提供 Zod Schema，用于配置验证：

```
export {
  DiscordConfigSchema,
  GoogleChatConfigSchema,
  IMessageConfigSchema,
  MSTeamsConfigSchema,
  SignalConfigSchema,
  SlackConfigSchema,
  TelegramConfigSchema,
} from "../config/zod-schema.providers-core.js";

export { WhatsAppConfigSchema } from "../config/zod-schema.providers-whatsapp.js";
export { LineConfigSchema } from "../line/config-schema.js";
```

这些 Schema 的导出意义重大——扩展插件通常需要在自己的配置验证中**复用**核心通道的配置结构。例如，一个基于 WhatsApp 的增强插件可以 `extends` WhatsApp 的 Schema。

（4）通道特定的辅助函数

SDK 为每个核心通道导出了一组辅助函数，让扩展可以复用核心通道的逻辑：

```
// Channel: Discord
export { listDiscordAccountIds, resolveDefaultDiscordAccountId,
         resolveDiscordAccount } from "../discord/accounts.js";
export { discordOnboardingAdapter } from "../channels/plugins/onboarding/discord.js";

// Channel: Telegram
export { listTelegramAccountIds, resolveTelegramAccount } from "../telegram/accounts.js";
export { telegramOnboardingAdapter } from "../channels/plugins/onboarding/telegram.js";

// Channel: WhatsApp
export { listWhatsAppAccountIds, resolveWhatsAppAccount } from "../web/accounts.js";
export { whatsappOnboardingAdapter } from "../channels/plugins/onboarding/whatsapp.js";

// Channel: LINE
export { createInfoCard, createListCard, createImageCard,
         createActionCard } from "../line/flex-templates.js";
export { processLineMessage, hasMarkdownToConvert } from "../line/markdown-to-line.js";
```

这种设计使得扩展插件不需要"重新发明轮子"——如果你在开发一个 WhatsApp Business 扩展，可以直接复用核心 WhatsApp 通道的账号解析和媒体加载逻辑。

（5）共享基础设施

跨通道通用的工具函数和类型：

```
// 消息操作工具
export { createActionGate, jsonResult, readNumberParam,
         readStringParam } from "../agents/tools/common.js";

// 允许列表管理
export { mergeAllowlist, summarizeMapping } from "../channels/allowlists/resolve-utils.js";

// 提及门控
export { resolveMentionGating } from "../channels/mention-gating.js";

// ACK 响应
export { shouldAckReaction, removeAckReactionAfterReply } from "../channels/ack-reactions.js";

// 媒体处理
export { detectMime, extensionForMime } from "../media/mime.js";
export { loadWebMedia } from "../web/media.js";

// 日志传输
export { registerLogTransport } from "../logging/logger.js";

// 诊断事件
export { emitDiagnosticEvent, isDiagnosticsEnabled,
         onDiagnosticEvent } from "../infra/diagnostic-events.js";
```

PluginRuntime：插件的运行时沙箱

SDK 中最庞大的类型是 `PluginRuntime` 。它是一个巨型对象，包含了插件在运行时可能需要的**所有核心函数引用**。我们来看它的分层结构：

```typescript
// src/plugins/runtime/types.ts（简化结构）
export type PluginRuntime = {
  version: string;               // OpenClaw 版本

  config: {
    loadConfig();                // 读取当前配置
    writeConfigFile();           // 写入配置文件
  };

  system: {
    enqueueSystemEvent();        // 入队系统事件
    runCommandWithTimeout();     // 执行外部命令（带超时）
  };

  media: {
    loadWebMedia();              // 加载网络媒体
    detectMime();                // MIME 类型检测
    getImageMetadata();          // 图片元数据
    resizeToJpeg();              // 图片压缩
  };

  tts: {
    textToSpeechTelephony();     // 语音合成
  };

  tools: {
    createMemoryGetTool();       // 记忆检索工具
    createMemorySearchTool();    // 记忆搜索工具
  };

  channel: {
    text: { /* 文本分块、命令检测、Markdown 转换 */ };
    reply: { /* 回复分发、信封格式化、人类延迟 */ };
    routing: { /* 路由解析 */ };
    pairing: { /* 配对管理 */ };
    media: { /* 媒体拉取与存储 */ };
    activity: { /* 通道活跃度记录 */ };
    session: { /* 会话存储操作 */ };
    mentions: { /* @提及匹配 */ };
    reactions: { /* 反应/表情管理 */ };
    groups: { /* 群组策略 */ };
    debounce: { /* 入站去抖 */ };
    commands: { /* 命令处理 */ };

    // 各核心通道的完整操作函数
    discord: { sendMessageDiscord(), monitorDiscordProvider(), ... };
    slack: { sendMessageSlack(), monitorSlackProvider(), ... };
    telegram: { sendMessageTelegram(), monitorTelegramProvider(), ... };
    signal: { sendMessageSignal(), monitorSignalProvider(), ... };
```

```
    imessage: { sendMessageIMessage(), monitorIMessageProvider(), ... };
    whatsapp: { sendMessageWhatsApp(), monitorWebChannel(), ... };
    line: { sendMessageLine(), pushFlexMessage(), ... };
  };

  logging: {
    shouldLogVerbose();          // 日志级别判断
    getChildLogger();            // 创建子日志器
  };

  state: {
    resolveStateDir();           // 解析状态存储目录
  };
};
```

> **衍生解释**：**Facade 模式**（外观模式）是一种结构型设计模式。它提供了一个简化的接口来访问一个复杂的子系统。 `PluginRuntime` 就是典型的 Facade——它将 OpenClaw 内部散布在数十个模块中的函数统一收拢到一个对象中，插件只需通过 `api.runtime.channel.telegram.sendMessageTelegram()` 就能调用原本深埋在内部模块中的功能。

`PluginRuntime` 的设计有一个重要特点：它包含了**所有核心通道**的操作函数。这意味着一个飞书扩展可以调用 `runtime.channel.telegram.sendMessageTelegram()` 来向 Telegram 发送消息。这种"跨通道操作"的能力为高级场景（如消息桥接、多通道联动）打开了大门。

21.2.3 插件钩子（Plugin Hooks）接口

钩子系统概览

OpenClaw 的插件钩子系统覆盖了消息处理链路上的所有关键节点。共有 **14 种钩子事件**，按生命周期域划分为五组：

钩子域	钩子名	执行模式	返回值
Agent	`before_agent_start`	顺序（串行）	可修改系统提示词
	`agent_end`	并行	无
	`before_compaction`	并行	无
	`after_compaction`	并行	无
Message	`message_received`	并行	无
	`message_sending`	顺序（串行）	可修改/取消消息
	`message_sent`	并行	无

钩子域	钩子名	执行模式	返回值
Tool	`before_tool_call`	顺序（串行）	可修改参数/阻止调用
	`after_tool_call`	并行	无
	`tool_result_persist`	同步串行	可修改持久化内容
Session	`session_start`	并行	无
	`session_end`	并行	无
Gateway	`gateway_start`	并行	无
	`gateway_stop`	并行	无

表中可以看到一个重要的规律：**只有能修改数据的钩子才使用串行执行**，纯通知性质的钩子全部并行执行以提高性能。

钩子的两种注册方式

插件有两种方式注册钩子：

方式一：通过 `registerHook()` 注册传统钩子

这是面向内部钩子系统（`src/hooks/`）的注册方式，兼容旧版本：

```
api.registerHook(
  "message_received",        // 事件名
  async (event) => {         // 处理函数
    console.log(`收到消息: ${event.content}`);
  },
  {
    name: "my-logger-hook",  // 钩子名称（必填）
    description: "日志记录收到的消息",
  }
);
```

方式二：通过 `on()` 注册类型安全的钩子

这是推荐的新方式，提供完整的 TypeScript 类型推导：

```
api.on("before_agent_start", async (event, ctx) => {
  // event: PluginHookBeforeAgentStartEvent（自动推导）
  // ctx: PluginHookAgentContext（自动推导）
  return {
    prependContext: "今天是周末，请使用轻松的语气。",
  };
}, { priority: 10 });
```

两种方式的区别：

维度	`registerHook()`	`on()`
类型安全	弱（handler 类型宽泛）	强（根据 hookName 推导）
注册目标	内部钩子系统 + 插件注册表	仅插件类型化注册表
优先级	通过 HookEntry 配置	通过 opts.priority 配置
推荐程度	兼容旧代码	✅ 推荐新代码使用

钩子运行器的实现

钩子的执行由 `createHookRunner()` 工厂函数创建的 `HookRunner` 负责。它内部有两种执行策略：

```
// src/plugins/hooks.ts
export function createHookRunner(registry: PluginRegistry, options = {}) {
  const catchErrors = options.catchErrors ?? true;

  // 并行执行：用于纯通知钩子
  async function runVoidHook(hookName, event, ctx) {
    const hooks = getHooksForName(registry, hookName);
    const promises = hooks.map(async (hook) => {
      try {
        await hook.handler(event, ctx);
      } catch (err) {
        if (catchErrors) logger?.error(msg);
        else throw new Error(msg, { cause: err });
      }
    });
    await Promise.all(promises);  // 所有处理函数并行执行
  }

  // 串行执行：用于可修改数据的钩子
  async function runModifyingHook(hookName, event, ctx, mergeResults?) {
    const hooks = getHooksForName(registry, hookName);
    let result;
    for (const hook of hooks) {   // 按优先级顺序串行执行
      const handlerResult = await hook.handler(event, ctx);
      if (handlerResult != null) {
        result = mergeResults ? mergeResults(result, handlerResult) : handlerResult;
      }
    }
    return result;
  }
  // ...
}
```

优先级排序的实现在 `getHooksForName()` 中：

```
function getHooksForName(registry, hookName) {
  return registry.typedHooks
    .filter((h) => h.hookName === hookName)
    .toSorted((a, b) => (b.priority ?? 0) - (a.priority ?? 0));  // 高优先级先执行
}
```

关键钩子详解

`before_agent_start` ：注入上下文

这是最强大的钩子之一，允许插件在 Agent 启动前修改系统提示词：

```
api.on("before_agent_start", async (event, ctx) => {
  // event.prompt — 当前系统提示词
  // event.messages — 历史消息
  return {
    systemPrompt: event.prompt + "\n你是一个企业客服机器人。",
    prependContext: "当前用户的 VIP 等级: 钻石",
  };
});
```

多个插件注册此钩子时，结果会被**合并**：

```
// hooks.ts — before_agent_start 的合并策略
runModifyingHook("before_agent_start", event, ctx, (acc, next) => ({
  systemPrompt: next.systemPrompt ?? acc?.systemPrompt,      // 后者覆盖
  prependContext: acc?.prependContext && next.prependContext
    ? `${acc.prependContext}\n\n${next.prependContext}`       // 拼接
    : (next.prependContext ?? acc?.prependContext),
}));
```

`message_sending` ：拦截/修改出站消息

允许插件在消息发送前修改内容或取消发送：

```
api.on("message_sending", async (event, ctx) => {
  // 敏感词过滤
  if (containsSensitiveWords(event.content)) {
    return { cancel: true };  // 取消发送
  }
  return {
    content: event.content.replace(/内部机密/g, "[已屏蔽]"),
  };
});
```

`before_tool_call`：工具调用拦截

允许插件修改工具参数或阻止工具执行：

```
api.on("before_tool_call", async (event, ctx) => {
  if (event.toolName === "web_search" && event.params.query?.includes("竞品")) {
    return { block: true, blockReason: "搜索竞品信息已被策略禁止" };
  }
});
```

`tool_result_persist`：工具结果持久化控制

这是唯一的**同步钩子**，因为它在会话转录追加的热路径上执行。处理函数不能返回 Promise：

```
// hooks.ts — 同步执行 + Promise 防护
function runToolResultPersist(event, ctx) {
  for (const hook of hooks) {
    const out = hook.handler({ ...event, message: current }, ctx);

    // 防护：如果处理函数错误地返回了 Promise，则忽略其结果
    if (out && typeof out.then === "function") {
      logger?.warn("tool_result_persist handler returned a Promise; this hook is sync-only");
      continue;
    }

    if (out?.message) current = out.message;
  }
  return { message: current };
}
```

全局钩子运行器

为了让代码库中任何位置都能触发钩子，OpenClaw 使用了**全局单例**模式：

```typescript
// src/plugins/hook-runner-global.ts
let globalHookRunner: HookRunner | null = null;

// 在插件加载时初始化
export function initializeGlobalHookRunner(registry: PluginRegistry): void {
  globalHookRunner = createHookRunner(registry, {
    logger: { debug: ..., warn: ..., error: ... },
    catchErrors: true,          // 生产环境中钩子异常不应崩溃主程序
  });
}

// 在任何位置获取
export function getGlobalHookRunner(): HookRunner | null {
  return globalHookRunner;
}

// 快速检查某个钩子是否有注册
export function hasGlobalHooks(hookName): boolean {
  return globalHookRunner?.hasHooks(hookName) ?? false;
}
```

`hasGlobalHooks()` 的存在是一个性能优化——调用方可以在构造钩子事件对象之前先检查是否有人监听，避免不必要的对象创建开销。

本节小结

1. **OpenClaw 的扩展 API 分为两层**： `extensionAPI.ts` 面向内置扩展，导出 Agent 运行时和会话存储接口； `plugin-sdk/index.ts` 面向第三方插件，导出完整的通道类型、配置 Schema 和辅助函数。
2. **Plugin SDK 是一个 375 行的再导出模块**，遵循"窄接口、宽实现"原则，导出内容分为通道适配器类型、插件核心类型、配置 Schema、通道辅助函数和共享基础设施五大类。
3. **`PluginRuntime` 是最庞大的运行时 Facade**，将分布在数十个内部模块中的功能统一收拢，并支持跨通道操作（如飞书插件调用 Telegram 发送函数）。
4. **钩子系统包含 14 种事件**，分为并行执行（纯通知）和串行执行（可修改）两种模式， `tool_result_persist` 是唯一的同步钩子。
5. **推荐使用 `on()` 方法注册钩子**，它提供完整的 TypeScript 类型推导；关键的修改型钩子（ `before_agent_start` 、 `message_sending` 、 `before_tool_call` ）支持结果合并策略。